

Goldman Sachs Internship Interview Experience

About Goldman Sachs:-

Goldman Sachs, founded in **1869** and headquartered in New York, is a leading global investment banking, securities, and investment management firm. It provides financial services to corporations, governments, and individuals worldwide, with a strong focus on technology, innovation, and sustainable finance. **In India**, the firm established its presence in **2004** and now operates from **Bengaluru, Hyderabad, and Mumbai**. The Bengaluru office, its largest outside New York, focuses on engineering, technology, and risk management, while Hyderabad drives innovation and Mumbai is the investment banking hub. The India offices play a strategic role in global decision-making, not just support, making them integral to the firm's growth. Goldman Sachs is also known for its strong culture of teamwork, leadership, and talent development, offering great opportunities for students.

Eligible Branches: UG: AE, AU, AIDS, BM, CE, CS, EE, EC, EI, GI, IE, IT, MN, MS, ME, MI, PT, PE, RP & RA

Eligibility Criteria: 6 & above; No current arrears

Title: Engineering Analyst Office

Location: Bengaluru and Hyderabad

Stipend : Rs.1,62,500

Internship Duration: 8 weeks

About Me:-

I am **Hariharan R I**, a Computer Science and Engineering student at Madras Institute of Technology, Anna University, with a **CGPA of 9.54**(then), and the department topper. I began practicing Data Structures and Algorithms (DSA) from my second semester on **Problem Solving section** of **HackerRank** and then later moved on to **LeetCode** from my third semester. Though I initially struggled with consistency during 3rd semester, I regained momentum in my fourth semester and have since been solving LeetCode problems daily, completing more than **500+** problems including the **LeetCode Top Interview 150** and **Striver's A2Z DSA sheet** on the time I attended GS Interview. Alongside, I explored full-stack and MERN stack web development during my first-year summer break, which greatly supported my fourth-semester Full-stack Technologies coursework. I also utilized my 1.5-month summer break after my second year for focused internship preparation and consistent DSA practice, which I consider as **Golden Time** in my Intern Preparation Journey, and advice all to prepare seriously during that time.

Online Assessment:-

The test was conducted on the **Hackerrank** platform on **27th July 2025**, and it was the very first online assessment we attended during the internship season.

- The test had **7 sections** with a total duration of **1 hour**.
- All sections were **mandatory**.
- **Coding Section 1** – 1 question, 10 mins.
- **Coding Section 2** – 1 question, 15 mins.
- **Coding Section 3** – 1 question, 15 mins.
- **MCQ Section 1** – 5 questions, 7 mins.
- **MCQ Section 2** – 2 questions, 3 mins.
- **MCQ Section 3** – 3 questions, 5 mins.
- **MCQ marking scheme** – +3 for correct, -1 for incorrect.
- **“Team Conflict HR” section** – 1 question, 5 mins.

The MCQs covered topics such as **OOPS, Java, Output, Debugging, Time Complexity Analysis, DBMS, OS concepts**, along with **Aptitude and Maths** (especially Linear Algebra) questions. Both the coding and MCQ sections carried **equal weightage**, which many participants were not aware of, so it is important to give your best in both sections instead of focusing only on one. The total duration was just one hour, making time management a key factor. Additionally, there was one HR-style question based on team conflict, which tested our situational judgment.

DSA Question-1 :-

Question

A robot starts at the top-left corner of an $m \times n$ factory floor grid and needs to reach the bottom-right corner. The robot can only move in two directions: either one step to the right or one step down. The task was to determine the total number of unique paths the robot can take to reach the destination. The input consisted of two integers representing the number of rows and columns, and the output was the total number of unique paths possible under the given movement constraints.

Approach

I identified this as a combinatorics problem. To reach the destination, the robot must make $(m-1)$ downward moves and $(n-1)$ rightward moves, i.e., $(m+n-2)$ total moves in any order. Thus, the problem reduces to computing the binomial coefficient $C(m+n-2, m-1)$. I implemented this in C++ using an iterative formula for

binomial coefficients to avoid overflow from factorials. This gave an efficient solution with **$O(\min(m, n))$** time complexity and **$O(1)$** space.

Result

In the First Attempt itself, the code had passed all the Test Cases and successfully submitted the question and moved on to next.

DSA Question-2 :-

Question

Another OA problem was about creating a balanced fertilizer mix. A farmer has **N organic substances**, each with two properties:

- **Growth Boost (G)** → a multiplicative factor that boosts plant growth
- **Soil Toxicity (T)** → an additive factor that harms soil quality

The goal was to select **at least one substance** such that the **absolute difference** between the **total growth boost** (product of selected G values) and the **total toxicity** (sum of selected T values) is minimized.

The input consisted of an integer N followed by N pairs of integers (G, T), and the output was the minimum possible difference across all valid combinations.

Approach

Since $N \leq 10$, I solved it using a **bitmask subset approach**.

- For each non-empty subset of substances, I computed:
 - ❖ **Total Growth** as the product of selected G values.
 - ❖ **Total Toxicity** as the sum of selected T values.
- I then tracked the **absolute difference** between them and updated the minimum difference.
- This works efficiently because there are only $2^N - 1$ subsets (at most 1023 when $N=10$).

The solution had **$O(2^N * N)$** time complexity (iterating through subsets and elements) and **$O(1)$** extra space.

Result

The Code had only passed 11 out of 12 Test Cases and due to time constraint, I later moved on to next question.

DSA Question-3 :-

Question

One of the OA problems was to implement a **mentor allotment system** in a consulting organization. Each employee is described by two attributes: **skill level (A)** and **project exposure (B)**. An employee can be mentored only by someone with **equal or higher skill and exposure**. Among all valid mentors, the one with the **smallest exposure difference** is chosen, and if there's a tie, the **smallest skill difference** is used. Each mentor can guide only one employee, and if a seeker re-queries, their previous mentor is permanently barred from mentoring anyone else. The input consisted of a series of operations: E A B when a new employee joins and Q i when the i-th employee requests mentorship. The task was to output the most suitable mentor's ID or "NE" if none exist.

Approach

Since the constraints allowed up to **22,000 operations**, a brute-force search for each query would be inefficient. To solve this, I used a **segment tree of sets**:

- Employees were stored in the tree based on skill levels, with each node maintaining a sorted set of {exposure, id} pairs.
- When a new employee joined, they were inserted into the tree. When an employee queried for a mentor, their data was temporarily removed to avoid self-selection.
- For queries, I searched in the range of skills $\geq$ seeker's skill and used **lower_bound** to quickly find the first mentor with exposure $\geq$ seeker's exposure. Tie-breaking was done using exposure first, then skill difference.
- Once a mentor was assigned, they were removed from the tree to ensure one-to-one matching. If the seeker re-queried, the old mentor was permanently blocked.

This approach ensured efficient handling of joins and queries in **$O(\log A)$** per operation, with overall complexity around **$O(N \log A)$** and space **$O(N)$** .

Result

I had attended this question after completing MCQs Section, So I had very short time for solving this and so I got stucked up with Runtime errors and had no time to debug, so I haven't completely attended this question and this is the hardest question of all the three.

Interview Rounds :-

After three days on **30 July 2025**, the interview shortlist was released, and **30 candidates** were selected for the next rounds—**19 from MIT** and **11 from CEG**. However, since the Appian interview took place before the Goldman Sachs interview, two of the shortlisted candidates had already been selected by Appian. As a result, only **28 candidates** attended the Goldman Sachs interview on **11th August 2025**.

There were **three** interview rounds in total—**two technical** rounds and **one resume-based technical + HR** round. The process was organized across three panels, with each panel handling a different round for a candidate. For example, if a candidate attended Round 1 in Panel A, then they would attend Round 2 in Panel B and Round 3 in Panel C. In my case, I attended Round 1 in Panel C, Round 2 in Panel B, and Round 3 in Panel A.

Interview Round 1 :-

Round 1 was conducted in two batches. **15 candidates** attended the **morning session**, while I was part of the **afternoon batch of 13 candidates**. In the morning batch, out of 15 participants, **4** were shortlisted for **Round 2**. During that time, the rest of us were asked to wait and instructed to go for lunch while those 4 attended their next round.

I attended my Round 1 interview at 3:00 PM, and it lasted for about 30 minutes. The interviewer began by asking me to **introduce myself**, where I highlighted my academic background, technical skills, and problem-solving experience. After this, he asked if I was familiar with Sudoku. I replied yes, and he then asked me to explain the rules of the game. Once I explained, he acknowledged my response and gave me a DSA problem based on it — **Valid Sudoku**, where the task was to check whether a given solved Sudoku is valid or not.

I explained my approach step by step: I planned to use **static 2D arrays** to check for duplicates in rows, columns, and 3x3 fields. For this, I created **three 9x9 arrays**, where each array tracked whether a number had already been used in a particular row, column, or field. I then demonstrated the logic, showing how I would store and check values in each structure. For the fields, I calculated the field number and applied the same duplicate-checking logic. Although the interviewer expected a set-based approach, I clarified that my static array method also worked correctly, and he was satisfied with my explanation.

Next, I discussed the time and space complexity of my solution: **$O(n^2)$** for iterating, storing, and checking, with space complexity of **$O(3 \cdot n^2)$** , which reduces to **$O(1)$** for a fixed-size Sudoku (9x9). As a follow-up, he asked about the **property of n** , to which I answered that **n** should be a **perfect square** number. Toward the end, he asked me if I had any questions for him, and I specifically asked about the **GS Legend platform and its role in open-source contributions**. He explained it briefly and concluded the round by

wishing me good luck for the next one. I came out feeling confident that I had performed well in this round and had been **selected for Round 2**.

Interview Round 2 :-

In the morning batch of Round 1, **4 candidates** were shortlisted for Round 2, and in the afternoon batch, **3 more were shortlisted**, including myself. My **Round 2 interview started around 4:15 PM and lasted for about 40 minutes**. At the beginning, the interviewer asked me to **introduce myself** and then requested me to **explain the projects** I had mentioned in my resume. I walked him through my projects, after which he asked a few simple clarifying questions on them.

The first DSA problem was the **Airport–Plane problem**: there are multiple airports connected by planes, with a cost associated with traveling between them. The task was to start from one airport, visit all airports exactly once, and return the **shortest possible cost**, while also explaining the data structure and algorithm to implement this. I immediately recognized this as the **Travelling Salesman Problem (TSP)**. I explained that the problem can be modeled as a **Graph**, where airports are nodes and planes with costs are weighted edges. I then described the TSP approach, explained its working, and wrote pseudocode for the solution. I also mentioned that the time complexity of the exact solution using dynamic programming is $O(n^2 \cdot 2^n)$, which becomes infeasible for larger values of n , and that approximate/heuristic methods are often used in practice. The interviewer was satisfied with my explanation and pseudocode.

The second problem was about a **server handling requests with a fixed buffer size**. If extra requests arrive when the buffer is full, the server should reply with a message that it is full. Each request is handled in a fixed time slot, and after each time slot, the next request is served. I identified this as a **Queue-based problem**. I explained that I would enqueue requests as they arrive, respond with a “server full” message if the buffer is at capacity, and dequeue a request after every time slot. The time complexity for both enqueue and dequeue operations is $O(1)$. Additionally, I optimized the dequeuing logic by maintaining a **time variable** and a **time array** that tracks which requests must be dequeued at each time slot. With this, the time pointer simply moves to the next slot for automatic dequeuing in $O(1)$. I further explained that we could even optimize by storing the next request’s dequeue time in the array itself. The interviewer appreciated this detailed explanation and said my logic was correct.

Toward the end, he asked if I had any questions. I asked about the **purpose behind setting up the Goldman Sachs Hyderabad office**, which was recently opened in 2021. He explained the reason and wished me all the best for the next round. I came out of the room feeling happy and confident that I would be shortlisted for the following round.

Interview Round 3 : -

In total, **7 people attended Round 2**, out of which **3 were shortlisted for Round 3** — one from the morning batch and two from the afternoon batch, and I was one among them. I attended my **Round 3 interview at 5:15 PM**, and it went on for about **35–40 minutes**.

The interviewer started by asking me to **introduce myself** and then moved to my **first project** from the resume, which was a **full-stack application with AI integration**. She asked:

- Where I had used AI in the project.
- How I had integrated AI features, and to explain in detail about some of the specific features I had mentioned.
- Whether encryption was required in the network layer, to which I explained various **cryptography techniques** that could be implemented in web development projects.

Next, she asked me about my **second full-stack project**. The questions included:

- Whether I had done any testing on it.
- If I had made improvements after testing.
- Questions on **database indexing**, where I also extended my answer by explaining **paging** and **lazy loading** techniques.

After that, she shifted to the **skills section** of my resume and asked:

- What I know about **Google Cloud** (I explained the areas I am most familiar with).
- Whether I use **Canva** for website design (I clarified that I used it mainly for **club work**, and explained its purpose).
- What **security features** I had implemented in my projects. I explained about **JWT, Firebase Authentication, and token-based authentication and authorization**.

She also asked a few more minor questions related to my resume. Toward the end, she asked if I had any questions for her. I asked about the **mentorship provided during GS internships** and the **hierarchy of roles** at Goldman Sachs. She answered my queries and then asked me to wait outside. That concluded my final Resume based technical + HR interview round. And I came out at around 6pm.

Aftermath:-

After all the interviews were completed, the three of us shortlisted for the final round waited together. We were a bit stressed and spent time discussing the questions asked in the previous rounds. At around **6:45 PM**, the **GS coordinator** (four people had come in total, including the three panelists) called us in and announced the results. To our delight, all three of us were **selected for the internship offer**. I was overjoyed to hear the news, and we were congratulated personally by the coordinators, followed by a short conversation with them.

Out of the three selected candidates, **one was from CSE at CEG**, while the other two were **me and my friend from CSE at MIT**. With immense happiness, we finally left **CUIC at around 7:20 PM**.

Personal Tips :-

- Know your resume well – this is the personal mantra.
- Practice DSA using LeetCode 150 and Striver's DSA Sheet.
- Engage with a senior from 2nd year onwards for intern preparation – in my case, a senior from my college guided me throughout.
- Communicate clearly – explain every step of your thinking while solving problems.
- Use your 2nd-year summer holidays wisely for dedicated internship preparation.
- Don't get demotivated if others move ahead before you. For example, some attended Round 2 in the morning while we were still waiting for Round 1 in the afternoon. Ultimately, it's not first-come-first-serve; they select the best performers, and most of the afternoon candidates were chosen in the end.
- Stay connected with your seniors and clear doubts regularly.
- For OOPS, DBMS, OS, and CN, attend college lectures sincerely – they cover enough to build a strong base. For further depth, refer to online resources and notes

For any Doubts or Queries, feel free to contact me:-

HARIHARAN R I (2023503555) – MIT Campus

+91 95973 53283

Mail – <mailto:hariharan17kpm@gmail.com>

LinkedIn - <https://www.linkedin.com/in/hariharan-r-i-507b1a304/>

GitHub - <https://github.com/Hack-Hero-17>